

第六章：几何

库实数类实现（双精度）

```
using Real = int;
using Point = complex<Real>;

Real cross(const Point &a, const Point &b) {
    return (conj(a) * b).imag();
}
Real dot(const Point &a, const Point &b) {
    return (conj(a) * b).real();
}
```

平面几何必要初始化

字符串读入浮点数

```
const int Knum = 4;
int read(int k = Knum) {
    string s;
    cin >> s;

    int num = 0;
    int it = s.find('.');
    if (it != -1) { // 存在小数点
        num = s.size() - it - 1; // 计算小数位数
        s.erase(s.begin() + it); // 删除小数点
    }
    for (int i = 1; i <= k - num; i++) { // 补全小数位数
        s += '0';
    }
    return stoi(s);
}
```

预置函数

```
using ld = long double;
const ld PI = acos(-1);
const ld EPS = 1e-7;
const ld INF = numeric_limits<ld>::max();
#define cc(x) cout << fixed << setprecision(x);

ld fgcd(ld x, ld y) { // 实数域gcd
    return abs(y) < EPS ? abs(x) : fgcd(y, fmod(x, y));
}
template<class T, class S> bool equal(T x, S y) {
    return -EPS < x - y && x - y < EPS;
}
template<class T> int sign(T x) {
```

```

    if (-EPS < x && x < EPS) return 0;
    return x < 0 ? -1 : 1;
}

```

点线封装

```

template<class T> struct Point { // 在C++17下使用 emplace_back 绑定可能会导致CE!
    T x, y;
    Point(T x_ = 0, T y_ = 0) : x(x_), y(y_) {} // 初始化
    template<class U> operator Point<U>() { // 自动类型匹配
        return Point<U>(U(x), U(y));
    }
    Point &operator+=(Point p) & { return x += p.x, y += p.y, *this; }
    Point &operator+=(T t) & { return x += t, y += t, *this; }
    Point &operator-=(Point p) & { return x -= p.x, y -= p.y, *this; }
    Point &operator-=(T t) & { return x -= t, y -= t, *this; }
    Point &operator*=(T t) & { return x *= t, y *= t, *this; }
    Point &operator/=(T t) & { return x /= t, y /= t, *this; }
    Point operator-() const { return Point(-x, -y); }
    friend Point operator+(Point a, Point b) { return a += b; }
    friend Point operator+(Point a, T b) { return a += b; }
    friend Point operator-(Point a, Point b) { return a -= b; }
    friend Point operator-(Point a, T b) { return a -= b; }
    friend Point operator*(Point a, T b) { return a *= b; }
    friend Point operator*(T a, Point b) { return b *= a; }
    friend Point operator/(Point a, T b) { return a /= b; }
    friend bool operator<(Point a, Point b) {
        return equal(a.x, b.x) ? a.y < b.y - EPS : a.x < b.x - EPS;
    }
    friend bool operator>(Point a, Point b) { return b < a; }
    friend bool operator==(Point a, Point b) { return !(a < b) && !(b < a); }
    friend bool operator!=(Point a, Point b) { return a < b || b < a; }
    friend auto &operator>>(istream &is, Point &p) {
        return is >> p.x >> p.y;
    }
    friend auto &operator<<(ostream &os, Point p) {
        return os << "(" << p.x << ", " << p.y << ")";
    }
};

template<class T> struct Line {
    Point<T> a, b;
    Line(Point<T> a_ = Point<T>(), Point<T> b_ = Point<T>()) : a(a_), b(b_) {}
    template<class U> operator Line<U>() { // 自动类型匹配
        return Line<U>(Point<U>(a), Point<U>(b));
    }
    friend auto &operator<<(ostream &os, Line l) {
        return os << "<" << l.a << ", " << l.b << ">";
    }
};

```

叉乘

定义公式 $a \times b = |a||b| \sin \theta$ 。

```
template<class T> T cross(Point<T> a, Point<T> b) { // 叉乘
    return a.x * b.y - a.y * b.x;
}
template<class T> T cross(Point<T> p1, Point<T> p2, Point<T> p0) { // 叉乘 (p1 - p0) x (p2 - p0);
    return cross(p1 - p0, p2 - p0);
}
```

点乘

定义公式 $a \times b = |a||b| \cos \theta$ 。

```
template<class T> T dot(Point<T> a, Point<T> b) { // 点乘
    return a.x * b.x + a.y * b.y;
}
template<class T> T dot(Point<T> p1, Point<T> p2, Point<T> p0) { // 点乘 (p1 - p0) * (p2 - p0);
    return dot(p1 - p0, p2 - p0);
}
```

欧几里得距离公式

最常用的距离公式。**需要注意**，开根号会丢失精度，如无强制要求，先不要开根号，留到最后一步一起开。

```
template <class T> ld dis(T x1, T y1, T x2, T y2) {
    ld val = (x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2);
    return sqrt(val);
}
template <class T> ld dis(Point<T> a, Point<T> b) {
    return dis(a.x, a.y, b.x, b.y);
}
```

曼哈顿距离公式

```
template <class T> T dis1(Point<T> p1, Point<T> p2) { // 曼哈顿距离公式
    return abs(p1.x - p2.x) + abs(p1.y - p2.y);
}
```

将向量转换为单位向量

```
Point<ld> standardize(Point<ld> vec) { // 转换为单位向量
    return vec / sqrt(vec.x * vec.x + vec.y * vec.y);
}
```

向量旋转

将当前向量移动至原点后顺时针旋转 90° ，即获取垂直于当前向量的、起点为原点的向量。在计算垂线时非常有用。例如，要想获取点 a 绕点 o 顺时针旋转 90° 后的点，可以这样书写代码：`auto ans = o + rotate(o, a);`；如果是逆时针旋转，那么只需更改符号即可：`auto ans = o - rotate(o, a);`。

```
template<class T> Point<T> rotate(Point<T> p1, Point<T> p2) { // 旋转
    Point<T> vec = p1 - p2;
    return {-vec.y, vec.x};
}
```

平面角度与弧度

弧度角度相互转换

```
1d toDeg(1d x) { // 弧度转角度
    return x * 180 / PI;
}
1d toArc(1d x) { // 角度转弧度
    return PI / 180 * x;
}
```

正弦定理

$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$ ，其中 R 为三角形外接圆半径；

余弦定理（已知三角形三边，求角）

$\cos C = \frac{a^2 + b^2 - c^2}{2ab}$, $\cos B = \frac{a^2 + c^2 - b^2}{2ac}$, $\cos A = \frac{b^2 + c^2 - a^2}{2bc}$ 。可以借此推导出三角形面积公式
 $S_{\triangle ABC} = \frac{ab \cdot \sin C}{2} = \frac{bc \cdot \sin A}{2} = \frac{ac \cdot \sin B}{2}$ 。

注意，计算格式是：由 b, c, a 三边求 $\angle A$ ；由 a, c, b 三边求 $\angle B$ ；由 a, b, c 三边求 $\angle C$ 。

```
1d angle(1d a, 1d b, 1d c) { // 余弦定理
    1d val = acos((a * a + b * b - c * c) / (2.0 * a * b)); // 计算弧度
    return val;
}
```

求两向量的夹角

能够计算 $[0^\circ, 180^\circ]$ 区间的角度。

```
1d angle(Point<1d> a, Point<1d> b) {
    1d val = abs(cross(a, b));
    return abs(atan2(val, a.x * b.x + a.y * b.y));
}
```

向量旋转任意角度

逆时针旋转，转换公式：
$$\begin{cases} x' = x \cos \theta - y \sin \theta \\ y' = x \sin \theta + y \cos \theta \end{cases}$$

```
Point<ld> rotate(Point<ld> p, ld rad) {  
    return {p.x * cos(rad) - p.y * sin(rad), p.x * sin(rad) + p.y * cos(rad)};  
}
```

点绕点旋转任意角度

逆时针旋转，转换公式：
$$\begin{cases} x' = (x_0 - x_1) \cos \theta + (y_0 - y_1) \sin \theta + x_1 \\ y' = (x_1 - x_0) \sin \theta + (y_0 - y_1) \cos \theta + y_1 \end{cases}$$

```
Point<ld> rotate(Point<ld> a, Point<ld> b, ld rad) {  
    ld x = (a.x - b.x) * cos(rad) + (a.y - b.y) * sin(rad) + b.x;  
    ld y = (b.x - a.x) * sin(rad) + (a.y - b.y) * cos(rad) + b.y;  
    return {x, y};  
}
```

平面点线相关

点是否在直线上（三点是否共线）

```
template<class T> bool onLine(Point<T> a, Point<T> b, Point<T> c) {  
    return sign(cross(b, a, c)) == 0;  
}  
  
template<class T> bool onLine(Point<T> p, Line<T> l) {  
    return onLine(p, l.a, l.b);  
}
```

点是否在向量（直线）左侧

需要注意，向量的方向会影响答案；点在向量上不视为在左侧。

```
template<class T> bool pointOnLineLeft(Pt p, Lt l) {  
    return cross(l.b, p, l.a) > 0;  
}
```

两点是否在直线同侧/异侧

```
template<class T> bool pointOnLineSide(Pt p1, Pt p2, Lt vec) {  
    T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);  
    return sign(val) == 1;  
}  
  
template<class T> bool pointNotOnLineSide(Pt p1, Pt p2, Lt vec) {  
    T val = cross(p1, vec.a, vec.b) * cross(p2, vec.a, vec.b);  
    return sign(val) == -1;  
}
```

两直线相交交点

在使用前需要先判断直线是否平行。

```
Pd lineIntersection(Ld l1, Ld l2) {  
    ld val = cross(l2.b - l2.a, l1.a - l2.a) / cross(l2.b - l2.a, l1.a - l1.b);  
    return l1.a + (l1.b - l1.a) * val;  
}
```

两直线是否平行/垂直/相同

```
template<class T> bool lineParallel(Lt p1, Lt p2) {  
    return sign(cross(p1.a - p1.b, p2.a - p2.b)) == 0;  
}  
template<class T> bool lineVertical(Lt p1, Lt p2) {  
    return sign(dot(p1.a - p1.b, p2.a - p2.b)) == 0;  
}  
template<class T> bool same(Line<T> l1, Line<T> l2) {  
    return lineParallel(Line{l1.a, l2.b}, {l1.b, l2.a}) &&  
           lineParallel(Line{l1.a, l2.a}, {l1.b, l2.b}) && lineParallel(l1, l2);  
}
```

点到直线的最近距离与最近点

```
pair<Pd, ld> pointToLine(Pd p, Ld l) {  
    Pd ans = lineIntersection({p, p + rotate(l.a, l.b)}, l);  
    return {ans, dis(p, ans)};  
}
```

如果只需要计算最近距离，下方的写法可以减少书写的代码量，效果一致。

```
template<class T> ld disPointToLine(Pt p, Lt l) {  
    ld ans = cross(p, l.a, l.b);  
    return abs(ans) / dis(l.a, l.b); // 面积除以底边长  
}
```

点是否在线段上

```
template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点也算作在直线上  
    return sign(cross(p, l.a, l.b)) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x) &&  
           min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);  
}  
template<class T> bool pointOnSegment(Pt p, Lt l) { // 端点不算  
    return pointOnSegment(p, l) && min(l.a.x, l.b.x) < p.x && p.x < max(l.a.x, l.b.x) &&  
           min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b.y);  
}
```

点到线段的最近距离与最近点

```
pair<Pd, Ld> pointToSegment(Pd p, Ld l) {
    if (sign(dot(p, l.b, l.a)) == -1) { // 特判到两端点的距离
        return {l.a, dis(p, l.a)};
    } else if (sign(dot(p, l.a, l.b)) == -1) {
        return {l.b, dis(p, l.b)};
    }
    return pointToLine(p, l);
}
```

点在直线上的投影点（垂足）

```
Pd project(Pd p, Ld l) { // 投影
    Pd vec = l.b - l.a;
    Ld r = dot(vec, p - l.a) / (vec.x * vec.x + vec.y * vec.y);
    return l.a + vec * r;
}
```

线段的中垂线

```
template<class T> Lt midSegment(Lt l) {
    Pt mid = (l.a + l.b) / 2; // 线段中点
    return {mid, mid + rotate(l.a, l.b)};
}
```

两线段是否相交及交点

该扩展版可以同时返回相交状态和交点，分为四种情况：0 代表不相交；1 代表普通相交；2 代表重叠（交于两个点）；3 代表相交于端点。**需要注意**，部分运算可能会使用到直线求交点，此时务必保证变量类型为浮点数！

```
template<class T> tuple<int, Pt, Pt> segmentIntersection(Lt l1, Lt l2) {
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
    auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
    auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
    auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
    if (A < CC || C < AA || B < DD || D < BB) {
        return {0, {}, {}};
    }
    if (sign(cross(e1 - s1, e2 - s2)) == 0) {
        if (sign(cross(s2, e1, s1)) != 0) {
            return {0, {}, {}};
        }
        Pt p1(max(AA, CC), max(BB, DD));
        Pt p2(min(A, C), min(B, D));
        if (!pointOnSegment(p1, l1)) {
            swap(p1.y, p2.y);
        }
        if (p1 == p2) {
```

```

        return {3, p1, p2};
    } else {
        return {2, p1, p2};
    }
}

auto cp1 = cross(s2 - s1, e2 - s1);
auto cp2 = cross(s2 - e1, e2 - e1);
auto cp3 = cross(s1 - s2, e1 - s2);
auto cp4 = cross(s1 - e2, e1 - e2);
if (sign(cp1 * cp2) == 1 || sign(cp3 * cp4) == 1) {
    return {0, {}, {}};
}
// 使用下方函数时请使用浮点数
Pd p = lineIntersection(l1, l2);
if (sign(cp1) != 0 && sign(cp2) != 0 && sign(cp3) != 0 && sign(cp4) != 0) {
    return {1, p, p};
} else {
    return {3, p, p};
}
}

```

如果不需要求交点，那么使用快速排斥+跨立实验即可，其中重叠、相交于端点均视为相交。

```

template<class T> bool segmentIntersection(Lt l1, Lt l2) {
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    auto A = max(s1.x, e1.x), AA = min(s1.x, e1.x);
    auto B = max(s1.y, e1.y), BB = min(s1.y, e1.y);
    auto C = max(s2.x, e2.x), CC = min(s2.x, e2.x);
    auto D = max(s2.y, e2.y), DD = min(s2.y, e2.y);
    return A >= CC && B >= DD && C >= AA && D >= BB &&
        sign(cross(s1, s2, e1) * cross(s1, e1, e2)) == 1 &&
        sign(cross(s2, s1, e2) * cross(s2, e2, e1)) == 1;
}

```

平面圆相关（浮点数处理）

点到圆的最近点

同时返回最近点与最近距离。**需要注意**，当点为圆心时，这样的点有无数个，此时我们视作输入错误，直接返回圆心。

```

pair<Pd, ld> pointToCircle(Pd p, Pd o, ld r) {
    Pd u = o, v = o;
    ld d = dis(p, o);
    if (sign(d) == 0) { // p 为圆心时返回圆心本身
        return {o, 0};
    }
    ld val1 = r * abs(o.x - p.x) / d;
    ld val2 = r * abs(o.y - p.y) / d * ((o.x - p.x) * (o.y - p.y) < 0 ? -1 : 1);
    u.x += val1, u.y += val2;
    v.x -= val1, v.y -= val2;
}

```



```

    if (dis(U, p) < dis(V, p)) {
        return {U, dis(U, p)};
    } else {
        return {V, dis(V, p)};
    }
}

```

根据圆心角获取圆上某点

将圆上最右侧的点以圆心为旋转中心，逆时针旋转 `rad` 度。

```

Point<ld> getPoint(Point<ld> p, ld r, ld rad) {
    return {p.x + cos(rad) * r, p.y + sin(rad) * r};
}

```

直线是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交。

```

tuple<int, Pd, Pd> lineCircleCross(Ld l, Pd o, ld r) {
    Pd P = project(o, l);
    ld d = dis(P, o), tmp = r * r - d * d;
    if (sign(tmp) == -1) {
        return {0, {}, {}};
    } else if (sign(tmp) == 0) {
        return {1, P, {}};
    }
    Pd vec = standardize(l.b - l.a) * sqrt(tmp);
    return {2, P + vec, P - vec};
}

```

线段是否与圆相交及交点

0 代表不相交；1 代表相切；2 代表相交于一个点；3 代表相交于两个点。

```

tuple<int, Pd, Pd> segmentCircleCross(Ld l, Pd o, ld r) {
    auto [type, U, V] = lineCircleCross(l, o, r);
    bool f1 = pointOnSegment(U, l), f2 = pointOnSegment(V, l);
    if (type == 1 && f1) {
        return {1, U, {}};
    } else if (type == 2 && f1 && f2) {
        return {3, U, V};
    } else if (type == 2 && f1) {
        return {2, U, {}};
    } else if (type == 2 && f2) {
        return {2, V, {}};
    } else {
        return {0, {}, {}};
    }
}

```

两圆是否相交及交点

0 代表内含；1 代表相离；2 代表相切；3 代表相交。

```
tuple<int, Pd, Pd> circleIntersection(Pd p1, ld r1, Pd p2, ld r2) {
    ld x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
    if (sign(abs(r1 - r2) - d) == 1) {
        return {0, {}, {}};
    } else if (sign(r1 + r2 - d) == -1) {
        return {1, {}, {}};
    }
    ld a = r1 * (x1 - x2) * 2, b = r1 * (y1 - y2) * 2, c = r2 * r2 - r1 * r1 - d * d;
    ld p = a * a + b * b, q = -a * c * 2, r = c * c - b * b;
    ld cosa, sina, cosb, sinb;
    if (sign(d - (r1 + r2)) == 0 || sign(d - abs(r1 - r2)) == 0) {
        cosa = -q / p / 2;
        sina = sqrt(1 - cosa * cosa);
        Point<ld> p0 = {x1 + r1 * cosa, y1 + r1 * sina};
        if (sign(dis(p0, p2) - r2)) {
            p0.y = y1 - r1 * sina;
        }
        return {2, p0, p0};
    } else {
        ld delta = sqrt(q * q - p * r * 4);
        cosa = (delta - q) / p / 2;
        cosb = (-delta - q) / p / 2;
        sina = sqrt(1 - cosa * cosa);
        sinb = sqrt(1 - cosb * cosb);
        Pd ans1 = {x1 + r1 * cosa, y1 + r1 * sina};
        Pd ans2 = {x1 + r1 * cosb, y1 + r1 * sinb};
        if (sign(dis(ans1, p1) - r2)) ans1.y = y1 - r1 * sina;
        if (sign(dis(ans2, p2) - r2)) ans2.y = y1 - r1 * sinb;
        if (ans1 == ans2) ans1.y = y1 - r1 * sina;
        return {3, ans1, ans2};
    }
}
```

两圆相交面积

上述所言四种相交情况均可计算，之所以不使用三角形面积计算公式是因为在计算过程中会出现“负数”面积（扇形面积与三角形面积的符号关系会随圆的位置关系发生变化），故公式全部重新推导，这里采用的是扇形面积减去扇形内部的那个三角形的面积。

```

1d circleIntersectionArea(Pd p1, 1d r1, Pd p2, 1d r2) {
    1d x1 = p1.x, x2 = p2.x, y1 = p1.y, y2 = p2.y, d = dis(p1, p2);
    if (sign(abs(r1 - r2) - d) >= 0) {
        return PI * min(r1 * r1, r2 * r2);
    } else if (sign(r1 + r2 - d) == -1) {
        return 0;
    }
    1d theta1 = angle(r1, dis(p1, p2), r2);
    1d area1 = r1 * r1 * (theta1 - sin(theta1 * 2) / 2);
    1d theta2 = angle(r2, dis(p1, p2), r1);
    1d area2 = r2 * r2 * (theta2 - sin(theta2 * 2) / 2);
    return area1 + area2;
}

```

三点确定一圆

```

tuple<int, Pd, 1d> getCircle(Pd A, Pd B, Pd C) {
    if (onLine(A, B, C)) { // 特判三点共线
        return {0, {}, 0};
    }
    Ld l1 = midSegment(Line{A, B});
    Ld l2 = midSegment(Line{A, C});
    Pd o = lineIntersection(l1, l2);
    return {1, o, dis(A, o)};
}

```

求解点到圆的切线数量与切点

```

pair<int, vector<Point<f64>>> tangent(Point<f64> A, Point<f64> O, f64 r) {
    vector<Point<f64>> ans; // 储存切点
    Point<f64> u = O - A;
    f64 d = sqrt(dot(u, u));
    if (d < r) {
        return {0, {}};
    } else if (sign(d - r) == 0) { // 点在圆上
        ans.push_back(u);
        return {1, ans};
    } else {
        A = O - u / d * r;
        f64 cos = r / d, sin = sqrt(1 - cos * cos);
        auto clac = [&]() -> Point<f64> {
            f64 x = A.x - O.x, y = A.y - O.y;
            return {x * cos + y * sin, (-x) * sin + y * cos};
        };
        ans.push_back(clac() + O);
        sin = -sin;
        ans.push_back(clac() + O);
        return {2, ans};
    }
}

```

求解两圆的内公、外公切线数量与切点

同时返回公切线数量以及每个圆的切点。

```
tuple<int, vector<Point<ld>>, vector<Point<ld>>> tangent(Point<ld> A, ld Ar, Point<ld> B, ld Br) {
    vector<Point<ld>> a, b; // 储存切点
    if (Ar < Br) {
        swap(Ar, Br);
        swap(A, B);
        swap(a, b);
    }
    int d = disEx(A, B), dif = Ar - Br, sum = Ar + Br;
    if (d < dif * dif) { // 内含, 无
        return {0, {}, {}};
    }
    ld base = atan2(B.y - A.y, B.x - A.x);
    if (d == 0 && Ar == Br) { // 完全重合, 无数条外公切线
        return {-1, {}, {}};
    }
    if (d == dif * dif) { // 内切, 1条外公切线
        a.push_back(getPoint(A, Ar, base));
        b.push_back(getPoint(B, Br, base));
        return {1, a, b};
    }
    ld ang = acos(dif / sqrt(d));
    a.push_back(getPoint(A, Ar, base + ang)); // 保底2条外公切线
    a.push_back(getPoint(A, Ar, base - ang));
    b.push_back(getPoint(B, Br, base + ang));
    b.push_back(getPoint(B, Br, base - ang));
    if (d == sum * sum) { // 外切, 多1条内公切线
        a.push_back(getPoint(A, Ar, base));
        b.push_back(getPoint(B, Br, base + PI));
    } else if (d > sum * sum) { // 相离, 多2条内公切线
        ang = acos(sum / sqrt(d));
        a.push_back(getPoint(A, Ar, base + ang));
        a.push_back(getPoint(A, Ar, base - ang));
        b.push_back(getPoint(B, Br, base + ang + PI));
        b.push_back(getPoint(B, Br, base - ang + PI));
    }
    return {a.size(), a, b};
}
```

平面三角形相关 (浮点数处理)

三角形面积

```
ld area(Point<ld> a, Point<ld> b, Point<ld> c) {
    return abs(cross(b, c, a)) / 2;
}
```

三角形外心

三角形外接圆的圆心，即三角形三边垂直平分线的交点。

```
template<class T> Pt center1(Pt p1, Pt p2, Pt p3) { // 外心
    return lineIntersection(midSegment({p1, p2}), midSegment({p2, p3}));
}
```

三角形内心

三角形内切圆的圆心，也是三角形三个内角的角平分线的交点。其到三角形三边的距离相等。

```
Pd center2(Pd p1, Pd p2, Pd p3) { // 内心
    #define atan2(p) atan2(p.y, p.x) // 注意先后顺序
    Line<ld> U = {p1, {}}, V = {p2, {}};
    ld m, n, alpha;
    m = atan2((p2 - p1));
    n = atan2((p3 - p1));
    alpha = (m + n) / 2;
    U.b = {p1.x + cos(alpha), p1.y + sin(alpha)};
    m = atan2((p1 - p2));
    n = atan2((p3 - p2));
    alpha = (m + n) / 2;
    V.b = {p2.x + cos(alpha), p2.y + sin(alpha)};
    return lineIntersection(U, V);
}
```

三角形垂心

三角形的三条高线所在直线的交点。锐角三角形的垂心在三角形内；直角三角形的垂心在直角顶点上；钝角三角形的垂心在三角形外。

```
Pd center3(Pd p1, Pd p2, Pd p3) { // 垂心
    Ld U = {p1, p1 + rotate(p2, p3)}; // 垂线
    Ld V = {p2, p2 + rotate(p1, p3)};
    return lineIntersection(U, V);
}
```

平面直线方程转换

浮点数计算直线的斜率

一般很少使用到这个函数，因为斜率的取值不可控（例如接近平行于 x, y 轴时）。**需要注意**，当直线平行于 y 轴时斜率为 `inf`。

```
template <class T> ld slope(Pt p1, Pt p2) { // 斜率，注意 inf 的情况
    return (p1.y - p2.y) / (p1.x - p2.x);
}
template <class T> ld slope(Lt l) {
    return slope(l.a, l.b);
}
```

分数精确计算直线的斜率

调用分数四则运算精确计算斜率，返回最简分数，只适用于整数计算。

```
template<class T> Frac<T> slopeEx(Pt p1, Pt p2) {
    Frac<T> u = p1.y - p2.y;
    Frac<T> v = p1.x - p2.x;
    return u / v; // 调用分数精确计算
}
```

两点式转一般式

返回由三个整数构成的方程，在输入较大时可能找不到较小的满足题意的一组整数解。可以处理平行于 x, y 轴、两点共点的情况。

```
template<class T> tuple<T, T, T> getfun(Lt p) {
    T A = p.a.y - p.b.y, B = p.b.x - p.a.x, C = p.a.x * A + p.a.y * B;
    if (A < 0) { // 符号调整
        A = -A, B = -B, C = -C;
    } else if (A == 0) {
        if (B < 0) {
            B = -B, C = -C;
        } else if (B == 0 && C < 0) {
            C = -C;
        }
    }
    if (A == 0) { // 数值计算
        if (B == 0) {
            C = 0; // 共点特判
        } else {
            T g = fgcd(abs(B), abs(C));
            B /= g, C /= g;
        }
    } else if (B == 0) {
        T g = fgcd(abs(A), abs(C));
        A /= g, C /= g;
    } else {
        T g = fgcd(fgcd(abs(A), abs(B)), abs(C));
        A /= g, B /= g, C /= g;
    }
    return tuple{A, B, C}; // Ax + By = C
}
```

一般式转两点式

由于整数点可能很大或者不存在，故直接采用浮点数；如果与 x, y 轴有交点则取交点。可以处理平行于 x, y 轴的情况。

```
Line<ld> getfun(int A, int B, int C) { // Ax + By = C
    ld x1 = 0, y1 = 0, x2 = 0, y2 = 0;
    if (A && B) { // 正常
        if (C) {
```

```

        x1 = 0, y1 = 1. * C / B;
        y2 = 0, x2 = 1. * C / A;
    } else { // 过原点
        x1 = 1, y1 = 1. * -A / B;
        x2 = 0, y2 = 0;
    }
} else if (A && !B) { // 垂直
    if (C) {
        y1 = 0, x1 = 1. * C / A;
        y2 = 1, x2 = x1;
    } else {
        x1 = 0, y1 = 1;
        x2 = 0, y2 = 0;
    }
} else if (!A && B) { // 水平
    if (C) {
        x1 = 0, y1 = 1. * C / B;
        x2 = 1, y2 = y1;
    } else {
        x1 = 1, y1 = 0;
        x2 = 0, y2 = 0;
    }
} else { // 不合法, 请特判
    assert(false);
}
return {{x1, y1}, {x2, y2}};
}

```

抛物线与 x 轴是否相交及交点

0 代表没有交点; 1 代表相切; 2 代表有两个交点。

```

tuple<int, ld, ld> getAns(ld a, ld b, ld c) {
    ld delta = b * b - a * c * 4;
    if (delta < 0.) {
        return {0, 0, 0};
    }
    delta = sqrt(delta);
    ld ans1 = -(delta + b) / 2 / a;
    ld ans2 = (delta - b) / 2 / a;
    if (ans1 > ans2) {
        swap(ans1, ans2);
    }
    if (sign(delta) == 0) {
        return {1, ans2, 0};
    }
    return {2, ans1, ans2};
}

```

平面多边形

两向量构成的平面四边形有向面积

```
template<class T> T areaEx(Point<T> p1, Point<T> p2, Point<T> p3) {  
    return cross(b, c, a);  
}
```

判断四个点能否组成矩形/正方形

可以处理浮点数、共点的情况。返回分为三种情况：2 代表构成正方形；1 代表构成矩形；0 代表其他情况。

```
template<class T> int issquare(vector<Pt> x) {  
    sort(x.begin(), x.end());  
    if (equal(dis(x[0], x[1]), dis(x[2], x[3])) && sign(dis(x[0], x[1])) &&  
        equal(dis(x[0], x[2]), dis(x[1], x[3])) && sign(dis(x[0], x[2])) &&  
        lineParallel(Lt{x[0], x[1]}, Lt{x[2], x[3]}) &&  
        lineParallel(Lt{x[0], x[2]}, Lt{x[1], x[3]}) &&  
        lineVertical(Lt{x[0], x[1]}, Lt{x[0], x[2]})) {  
        return equal(dis(x[0], x[1]), dis(x[0], x[2])) ? 2 : 1;  
    }  
    return 0;  
}
```

点是否在任意多边形内

射线法判定， t 为穿越次数，当其为奇数时即代表点在多边形内部；返回 2 代表点在多边形边界上。

```
template<class T> int pointInPolygon(Point<T> a, vector<Point<T>> p) {  
    int n = p.size();  
    for (int i = 0; i < n; i++) {  
        if (pointOnSegment(a, Line{p[i], p[(i + 1) % n]})) {  
            return 2;  
        }  
    }  
    int t = 0;  
    for (int i = 0; i < n; i++) {  
        auto u = p[i], v = p[(i + 1) % n];  
        if (u.x < a.x && v.x >= a.x && pointOnLineLeft(a, Line{v, u})) {  
            t ^= 1;  
        }  
        if (u.x >= a.x && v.x < a.x && pointOnLineLeft(a, Line{u, v})) {  
            t ^= 1;  
        }  
    }  
    return t == 1;  
}
```


线段是否在任意多边形内部

```
template<class T>
bool segmentInPolygon(Line<T> l, vector<Point<T>> p) {
    // 线段与多边形边界不相交且两端点都在多边形内部
#define L(x, y) pointOnLineLeft(x, y)
    int n = p.size();
    if (!pointInPolygon(l.a, p)) return false;
    if (!pointInPolygon(l.b, p)) return false;
    for (int i = 0; i < n; i++) {
        auto u = p[i];
        auto v = p[(i + 1) % n];
        auto w = p[(i + 2) % n];
        auto [t, p1, p2] = segmentIntersection(l, Line(u, v));
        if (t == 1) return false;
        if (t == 0) continue;
        if (t == 2) {
            if (pointOnSegment(v, l) && v != l.a && v != l.b) {
                if (cross(v - u, w - v) > 0) {
                    return false;
                }
            }
        }
    }
    else {
        if (p1 != u && p1 != v) {
            if (L(l.a, Line(v, u)) || L(l.b, Line(v, u))) {
                return false;
            }
        }
        else if (p1 == v) {
            if (l.a == v) {
                if (L(u, l)) {
                    if (L(w, l) && L(w, Line(u, v))) {
                        return false;
                    }
                }
            }
            else {
                if (L(w, l) || L(w, Line(u, v))) {
                    return false;
                }
            }
        }
        else if (l.b == v) {
            if (L(u, Line(l.b, l.a))) {
                if (L(w, Line(l.b, l.a)) && L(w, Line(u, v))) {
                    return false;
                }
            }
            else {
                if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
                    return false;
                }
            }
        }
        else {
            if (L(u, l)) {
                if (L(w, Line(l.b, l.a)) || L(w, Line(u, v))) {
                    return false;
                }
            }
        }
    }
}
```

```

        } else {
            if (L(w, l) || L(w, Line(u, v))) {
                return false;
            }
        }
    }
}
return true;
}

```

任意多边形的面积

```

template<class T> ld area(vector<Point<T>> P) {
    int n = P.size();
    ld ans = 0;
    for (int i = 0; i < n; i++) {
        ans += cross(P[i], P[(i + 1) % n]);
    }
    return ans / 2.0;
}

```

皮克定理

绘制在方格纸上的多边形面积公式可以表示为 $S = n + \frac{s}{2} - 1$ ，其中 n 表示多边形内部的点数、 s 表示多边形边界上的点数。一条线段上的点数为 $\gcd(|x_1 - x_2|, |y_1 - y_2|) + 1$ 。

任意多边形上/内的网格点个数（仅能处理整数）

皮克定理用。

```

int onPolygonGrid(vector<Point<int>> p) { // 多边形上
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; i++) {
        auto a = p[i], b = p[(i + 1) % n];
        ans += gcd(abs(a.x - b.x), abs(a.y - b.y));
    }
    return ans;
}

int inPolygonGrid(vector<Point<int>> p) { // 多边形内
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; i++) {
        auto a = p[i], b = p[(i + 1) % n], c = p[(i + 2) % n];
        ans += b.y * (a.x - c.x);
    }
    ans = abs(ans);
    return (ans - onPolygonGrid(p)) / 2 + 1;
}

```

二维凸包

获取二维静态凸包 (Andrew算法)

`flag` 用于判定凸包边上的点、重复的顶点是否要加入到凸包中，为 0 时代表加入凸包（不严格）；为 1 时不加入凸包（严格）。时间复杂度为 $\mathcal{O}(N \log N)$ 。

```
template<class T> vector<Point<T>> staticConvexHull(vector<Point<T>> A, int flag = 1) {
    int n = A.size();
    if (n <= 2) { // 特判
        return A;
    }
    vector<Point<T>> ans(n * 2);
    sort(A.begin(), A.end());
    int now = -1;
    for (int i = 0; i < n; i++) { // 维护下凸包
        while (now > 0 && cross(A[i], ans[now], ans[now - 1]) <= 0) {
            now--;
        }
        ans[++now] = A[i];
    }
    int pre = now;
    for (int i = n - 2; i >= 0; i--) { // 维护上凸包
        while (now > pre && cross(A[i], ans[now], ans[now - 1]) <= 0) {
            now--;
        }
        ans[++now] = A[i];
    }
    ans.resize(now);
    return ans;
}
```

二维动态凸包

固定为 `int` 型，需要重新书写 `Line` 函数，`cmp` 用于判定边界情况。可以处理如下两个要求：

- 动态插入点 (x, y) 到当前凸包中；
- 判断点 (x, y) 是否在凸包上或是在内部（包括边界）。

```
template<class T> bool turnRight(Pt a, Pt b) {
    return cross(a, b) < 0 || (cross(a, b) == 0 && dot(a, b) < 0);
}
struct Line {
    static int cmp;
    mutable Point<int> a, b;
    friend bool operator<(Line x, Line y) {
        return cmp ? x.a < y.a : turnRight(x.b, y.b);
    }
    friend auto &operator<<(ostream &os, Line l) {
        return os << "<" << l.a << ", " << l.b << ">";
    }
};
```

```

int Line::cmp = 1;
struct UpperConvexHull : set<Line> {
    bool contains(const Point<int> &p) const {
        auto it = lower_bound({p, 0});
        if (it != end() && it->a == p) return true;
        if (it != begin() && it != end() && cross(prev(it)->b, p - prev(it)->a) <= 0) {
            return true;
        }
        return false;
    }
    void add(const Point<int> &p) {
        if (contains(p)) return;
        auto it = lower_bound({p, 0});
        for (; it != end(); it = erase(it)) {
            if (turnRight(it->a - p, it->b)) {
                break;
            }
        }
        for (; it != begin() && prev(it) != begin(); erase(prev(it))) {
            if (turnRight(prev(prev(it))->b, p - prev(prev(it))->a)) {
                break;
            }
        }
        if (it != begin()) {
            prev(it)->b = p - prev(it)->a;
        }
        if (it == end()) {
            insert({p, {0, -1}});
        } else {
            insert({p, it->a - p});
        }
    }
};

struct ConvexHull {
    UpperConvexHull up, low;
    bool empty() const {
        return up.empty();
    }
    bool contains(const Point<int> &p) const {
        Line::cmp = 1;
        return up.contains(p) && low.contains(-p);
    }
    void add(const Point<int> &p) {
        Line::cmp = 1;
        up.add(p);
        low.add(-p);
    }
    bool isIntersect(int A, int B, int C) const {
        Line::cmp = 0;
        if (empty()) return false;
        Point<int> k = {-B, A};
        if (k.x < 0) k = -k;
    }
};

```

```

        if (k.x == 0 && k.y < 0) k.y = -k.y;
        Point<int> P = up.upper_bound({{0, 0}, k})->a;
        Point<int> Q = -low.upper_bound({{0, 0}, k})->a;
        return sign(A * P.x + B * P.y - C) * sign(A * Q.x + B * Q.y - C) > 0;
    }
    friend ostream &operator<<(ostream &out, const ConvexHull &ch) {
        for (const auto &line : ch.up) out << "(" << line.a.x << "," << line.a.y << ")";
        cout << "/";
        for (const auto &line : ch.low) out << "(" << -line.a.x << "," << -line.a.y << ")";
        return out;
    }
};

```

点与凸包的位置关系

0 代表点在凸包外面；1 代表在凸壳上；2 代表在凸包内部。

```

template<class T> int contains(Point<T> p, vector<Point<T>> A) {
    int n = A.size();
    bool in = false;
    for (int i = 0; i < n; i++) {
        Point<T> a = A[i] - p, b = A[(i + 1) % n] - p;
        if (a.y > b.y) {
            swap(a, b);
        }
        if (a.y <= 0 && 0 < b.y && cross(a, b) < 0) {
            in = !in;
        }
        if (cross(a, b) == 0 && dot(a, b) <= 0) {
            return 1;
        }
    }
    return in ? 2 : 0;
}

```

闵可夫斯基和

计算两个凸包合成的大凸包。

```

template<class T> vector<Point<T>> mincowski(vector<Point<T>> P1, vector<Point<T>> P2) {
    int n = P1.size(), m = P2.size();
    vector<Point<T>> V1(n), V2(m);
    for (int i = 0; i < n; i++) {
        V1[i] = P1[(i + 1) % n] - P1[i];
    }
    for (int i = 0; i < m; i++) {
        V2[i] = P2[(i + 1) % m] - P2[i];
    }
    vector<Point<T>> ans = {P1.front() + P2.front()};
    int t = 0, i = 0, j = 0;
    while (i < n && j < m) {
        Point<T> val = sign(cross(V1[i], V2[j])) > 0 ? V1[i++] : V2[j++];
    }
}

```

```

        ans.push_back(ans.back() + val);
    }
    while (i < n) ans.push_back(ans.back() + v1[i++]);
    while (j < m) ans.push_back(ans.back() + v2[j++]);
    return ans;
}

```

半平面交

计算多条直线左边平面部分的交集。

```

template<class T> vector<Point<T>> halfcut(vector<Line<T>> lines) {
    sort(lines.begin(), lines.end(), [&](auto l1, auto l2) {
        auto d1 = l1.b - l1.a;
        auto d2 = l2.b - l2.a;
        if (sign(d1) != sign(d2)) {
            return sign(d1) == 1;
        }
        return cross(d1, d2) > 0;
    });
    deque<Line<T>> ls;
    deque<Point<T>> ps;
    for (auto l : lines) {
        if (ls.empty()) {
            ls.push_back(l);
            continue;
        }
        while (!ps.empty() && !pointOnLineLeft(ps.back(), l)) {
            ps.pop_back();
            ls.pop_back();
        }
        while (!ps.empty() && !pointOnLineLeft(ps[0], l)) {
            ps.pop_front();
            ls.pop_front();
        }
        if (cross(l.b - l.a, ls.back().b - ls.back().a) == 0) {
            if (dot(l.b - l.a, ls.back().b - ls.back().a) > 0) {
                if (!pointOnLineLeft(ls.back().a, l)) {
                    assert(ls.size() == 1);
                    ls[0] = l;
                }
                continue;
            }
            return {};
        }
        ps.push_back(lineIntersection(ls.back(), l));
        ls.push_back(l);
    }
    while (!ps.empty() && !pointOnLineLeft(ps.back(), ls[0])) {
        ps.pop_back();
        ls.pop_back();
    }
    if (ls.size() <= 2) {

```

```

        return {};
    }
    ps.push_back(lineIntersection(ls[0], ls.back()));
    return vector(ps.begin(), ps.end());
}

```

三维几何必要初始化

点线面封装

```

struct Point3 {
    ld x, y, z;
    Point3(ld x_ = 0, ld y_ = 0, ld z_ = 0) : x(x_), y(y_), z(z_) {}
    Point3 &operator+=(Point3 p) & {
        return x += p.x, y += p.y, z += p.z, *this;
    }
    Point3 &operator-=(Point3 p) & {
        return x -= p.x, y -= p.y, z -= p.z, *this;
    }
    Point3 &operator*=(Point3 p) & {
        return x *= p.x, y *= p.y, z *= p.z, *this;
    }
    Point3 &operator*=(ld t) & {
        return x *= t, y *= t, z *= t, *this;
    }
    Point3 &operator/=(ld t) & {
        return x /= t, y /= t, z /= t, *this;
    }
    friend Point3 operator+(Point3 a, Point3 b) { return a += b; }
    friend Point3 operator-(Point3 a, Point3 b) { return a -= b; }
    friend Point3 operator*(Point3 a, Point3 b) { return a *= b; }
    friend Point3 operator*(Point3 a, ld b) { return a *= b; }
    friend Point3 operator*(ld a, Point3 b) { return b *= a; }
    friend Point3 operator/(Point3 a, ld b) { return a /= b; }
    friend auto &operator>>(istream &is, Point3 &p) {
        return is >> p.x >> p.y >> p.z;
    }
    friend auto &operator<<(ostream &os, Point3 p) {
        return os << "(" << p.x << ", " << p.y << ", " << p.z << ")";
    }
};

struct Line3 {
    Point3 a, b;
};

struct Plane {
    Point3 u, v, w;
};

```

其他函数

```
1d len(P3 p) { // 原点到当前点的距离计算
    return sqrt(p.x * p.x + p.y * p.y + p.z * p.z);
}
P3 crossEx(P3 a, P3 b) { // 叉乘
    P3 ans;
    ans.x = a.y * b.z - a.z * b.y;
    ans.y = a.z * b.x - a.x * b.z;
    ans.z = a.x * b.y - a.y * b.x;
    return ans;
}
1d cross(P3 a, P3 b) {
    return len(crossEx(a, b));
}
1d dot(P3 a, P3 b) { // 点乘
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
P3 getVec(Plane s) { // 获取平面法向量
    return crossEx(s.u - s.v, s.v - s.w);
}
1d dis(P3 a, P3 b) { // 三维欧几里得距离公式
    1d val = (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y) + (a.z - b.z) * (a.z - b.z);
    return sqrt(val);
}
P3 standardize(P3 vec) { // 将三维向量转换为单位向量
    return vec / len(vec);
}
```

三维点线面相关

空间三点是否共线

其中第二个函数是专门用来判断给定的三个点能否构成平面的，因为不共线的三点才能构成平面。

```
bool onLine(P3 p1, P3 p2, P3 p3) { // 三点是否共线
    return sign(cross(p1 - p2, p3 - p2)) == 0;
}
bool onLine(Plane s) {
    return onLine(s.u, s.v, s.w);
}
```

四点是否共面

```
bool onPlane(P3 p1, P3 p2, P3 p3, P3 p4) { // 四点是否共面
    1d val = dot(getVec({p1, p2, p3}), p4 - p1);
    return sign(val) == 0;
}
```


空间点是否在线段上

```
bool pointOnSegment(P3 p, L3 l) {
    return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) <= p.x &&
        p.x <= max(l.a.x, l.b.x) && min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y)
    &&
        min(l.a.z, l.b.z) <= p.z && p.z <= max(l.a.z, l.b.z);
}
bool pointOnSegmentEx(P3 p, L3 l) { // pointOnSegment去除端点版
    return sign(cross(p - l.a, p - l.b)) == 0 && min(l.a.x, l.b.x) < p.x &&
        p.x < max(l.a.x, l.b.x) && min(l.a.y, l.b.y) < p.y && p.y < max(l.a.y, l.b.y) &&
        min(l.a.z, l.b.z) < p.z && p.z < max(l.a.z, l.b.z);
}
```

空间两点是否在线段同侧

当给定的两点与线段不共面、点在线段上时返回 *false* 。

```
bool pointOnSegmentsSide(P3 p1, P3 p2, L3 l) {
    if (!onPlane(p1, p2, l.a, l.b)) { // 特判不共面
        return 0;
    }
    ld val = dot(crossEx(l.a - l.b, p1 - l.b), crossEx(l.a - l.b, p2 - l.b));
    return sign(val) == 1;
}
```

两点是否在平面同侧

点在平面上时返回 *false* 。

```
bool pointOnPlaneSide(P3 p1, P3 p2, Plane s) {
    ld val = dot(getVec(s), p1 - s.u) * dot(getVec(s), p2 - s.u);
    return sign(val) == 1;
}
```

空间两直线是否平行/垂直

```
bool lineParallel(L3 l1, L3 l2) {
    return sign(cross(l1.a - l1.b, l2.a - l2.b)) == 0;
}
bool lineVertical(L3 l1, L3 l2) {
    return sign(dot(l1.a - l1.b, l2.a - l2.b)) == 0;
}
```

两平面是否平行/垂直

```
bool planeParallel(Plane s1, Plane s2) {
    Id val = cross(getVec(s1), getVec(s2));
    return sign(val) == 0;
}
bool planeVertical(Plane s1, Plane s2) {
    Id val = dot(getVec(s1), getVec(s2));
    return sign(val) == 0;
}
```

空间两直线是否是同一条

```
bool same(L3 l1, L3 l2) {
    return lineParallel(l1, l2) && lineParallel({l1.a, l2.b}, {l1.b, l2.a});
}
```

两平面是否是同一个

```
bool same(Plane s1, Plane s2) {
    return onPlane(s1.u, s2.u, s2.v, s2.w) && onPlane(s1.v, s2.u, s2.v, s2.w) &&
        onPlane(s1.w, s2.u, s2.v, s2.w);
}
```

直线是否与平面平行

```
bool linePlaneParallel(L3 l, Plane s) {
    Id val = dot(l.a - l.b, getVec(s));
    return sign(val) == 0;
}
```

空间两线段是否相交

```
bool segmentIntersection(L3 l1, L3 l2) { // 重叠、相交于端点均视为相交
    if (!onPlane(l1.a, l1.b, l2.a, l2.b)) { // 特判不共面
        return 0;
    }
    if (!onLine(l1.a, l1.b, l2.a) || !onLine(l1.a, l1.b, l2.b)) {
        return !pointOnSegmentSide(l1.a, l1.b, l2) && !pointOnSegmentSide(l2.a, l2.b, l1);
    }
    return pointOnSegment(l1.a, l2) || pointOnSegment(l1.b, l2) || pointOnSegment(l2.a, l1)
        ||
        pointOnSegment(l2.b, l2);
}
bool segmentIntersection1(L3 l1, L3 l2) { // 重叠、相交于端点不视为相交
    return onPlane(l1.a, l1.b, l2.a, l2.b) && !pointOnSegmentSide(l1.a, l1.b, l2) &&
        !pointOnSegmentSide(l2.a, l2.b, l1);
}
```

空间两直线是否相交及交点

当两直线不共面、两直线平行时返回 *false* 。

```
pair<bool, P3> lineIntersection(L3 l1, L3 l2) {
    if (!onPlane(l1.a, l1.b, l2.a, l2.b) || lineParallel(l1, l2)) {
        return {0, {}};
    }
    auto [s1, e1] = l1;
    auto [s2, e2] = l2;
    ld val = 0;
    if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 0, 1})) {
        val = ((s1.x - s2.x) * (s2.y - e2.y) - (s1.y - s2.y) * (s2.x - e2.x)) /
              ((s1.x - e1.x) * (s2.y - e2.y) - (s1.y - e1.y) * (s2.x - e2.x));
    } else if (!onPlane(l1.a, l1.b, {0, 0, 0}, {0, 1, 0})) {
        val = ((s1.x - s2.x) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.x - e2.x)) /
              ((s1.x - e1.x) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.x - e2.x));
    } else {
        val = ((s1.y - s2.y) * (s2.z - e2.z) - (s1.z - s2.z) * (s2.y - e2.y)) /
              ((s1.y - e1.y) * (s2.z - e2.z) - (s1.z - e1.z) * (s2.y - e2.y));
    }
    return {1, s1 + (e1 - s1) * val};
}
```

直线与平面是否相交及交点

当直线与平面平行、给定的点构不成平面时返回 *false* 。

```
pair<bool, P3> linePlaneCross(L3 l, Plane s) {
    if (linePlaneParallel(l, s)) {
        return {0, {}};
    }
    P3 vec = getVec(s);
    P3 U = vec * (s.u - l.a), V = vec * (l.b - l.a);
    ld val = (U.x + U.y + U.z) / (V.x + V.y + V.z);
    return {1, l.a + (l.b - l.a) * val};
}
```

两平面是否相交及交线

当两平面平行、两平面为同一个时返回 *false* 。

```
pair<bool, L3> planeIntersection(Plane s1, Plane s2) {
    if (planeParallel(s1, s2) || same(s1, s2)) {
        return {0, {}};
    }
    P3 U = linePlaneParallel({s2.u, s2.v}, s1) ? linePlaneCross({s2.v, s2.w}, s1).second
                                                : linePlaneCross({s2.u, s2.v}, s1).second;
    P3 V = linePlaneParallel({s2.w, s2.u}, s1) ? linePlaneCross({s2.v, s2.w}, s1).second
                                                : linePlaneCross({s2.w, s2.u}, s1).second;
    return {1, {U, V}};
}
```

点到直线的最近点与最近距离

```
pair<ld, P3> pointToLine(P3 p, L3 l) {
    ld val = cross(p - l.a, l.a - l.b) / dis(l.a, l.b); // 面积除以底边长
    ld val1 = dot(p - l.a, l.a - l.b) / dis(l.a, l.b);
    return {val, l.a + val1 * standardize(l.a - l.b)};
}
```

点到平面的最近点与最近距离

```
pair<ld, P3> pointToPlane(P3 p, Plane s) {
    P3 vec = getVec(s);
    ld val = dot(vec, p - s.u);
    val = abs(val) / len(vec); // 面积除以底边长
    return {val, p - val * standardize(vec)};
}
```

空间两直线的最近距离与最近点对

```
tuple<ld, P3, P3> lineToLine(L3 l1, L3 l2) {
    P3 vec = crossEx(l1.a - l1.b, l2.a - l2.b); // 计算同时垂直于两直线的向量
    ld val = abs(dot(l1.a - l2.a, vec)) / len(vec);
    P3 u = l1.b - l1.a, v = l2.b - l2.a;
    vec = crossEx(u, v);
    ld p = dot(vec, vec);
    ld t1 = dot(crossEx(l2.a - l1.a, v), vec) / p;
    ld t2 = dot(crossEx(l2.a - l1.a, u), vec) / p;
    return {val, l1.a + (l1.b - l1.a) * t1, l2.a + (l2.b - l2.a) * t2};
}
```

三维角度与弧度

空间两直线夹角的 cos 值

任意位置的空间两直线。

```
ld lineCos(L3 l1, L3 l2) {
    return dot(l1.a - l1.b, l2.a - l2.b) / len(l1.a - l1.b) / len(l2.a - l2.b);
}
```

空间两平面夹角的 cos 值

```
ld planeCos(Plane s1, Plane s2) {
    P3 u = getVec(s1), v = getVec(s2);
    return dot(u, v) / len(u) / len(v);
}
```

直线与平面夹角的 sin 值

```
1d linePlaneSin(L3 l, Plane s) {
    P3 vec = getVec(s);
    return dot(l.a - l.b, vec) / len(l.a - l.b) / len(vec);
}
```

空间多边形

正N棱锥体积公式

棱锥通用体积公式 $V = \frac{1}{3}Sh$ ，当其恰好是棱长为 l 的正 n 棱锥时，有公式 $V = \frac{l^3 \cdot n}{12 \tan \frac{\pi}{n}} \cdot \sqrt{1 - \frac{1}{4 \cdot \sin^2 \frac{\pi}{n}}}$ 。

```
1d V(1d l, int n) { // 正n棱锥体积公式
    return l * l * l * n / (12 * tan(PI / n)) * sqrt(1 - 1 / (4 * sin(PI / n) * sin(PI /
n)));
}
```

四面体体积

```
1d V(P3 a, P3 b, P3 c, P3 d) {
    return abs(dot(d - a, crossEx(b - a, c - a))) / 6;
}
```

点是否在空间三角形上

点位于边界上时返回 *false*。

```
bool pointOnTriangle(P3 p, P3 p1, P3 p2, P3 p3) {
    return pointOnSegmentSide(p, p1, {p2, p3}) && pointOnSegmentSide(p, p2, {p1, p3}) &&
        pointOnSegmentSide(p, p3, {p1, p2});
}
```

线段是否与空间三角形相交及交点

只有交点在空间三角形内部时才视作相交。

```
pair<bool, P3> segmentOnTriangle(P3 l, P3 r, P3 p1, P3 p2, P3 p3) {
    P3 x = crossEx(p2 - p1, p3 - p1);
    if (sign(dot(x, r - l)) == 0) {
        return {0, {}};
    }
    1d t = dot(x, p1 - l) / dot(x, r - l);
    if (t < 0 || t - 1 > 0) { // 不在线段上
        return {0, {}};
    }
    bool type = pointOnTriangle(l + (r - l) * t, p1, p2, p3);
}
```

```
    if (type) {
        return {1, 1 + (r - 1) * t};
    } else {
        return {0, {}};
    }
}
```

空间三角形是否相交

相交线段在空间三角形内部时才视作相交。

```
bool triangleIntersection(vector<P3> a, vector<P3> b) {
    for (int i = 0; i < 3; i++) {
        if (segmentOnTriangle(b[i], b[(i + 1) % 3], a[0], a[1], a[2]).first) {
            return 1;
        }
        if (segmentOnTriangle(a[i], a[(i + 1) % 3], b[0], b[1], b[2]).first) {
            return 1;
        }
    }
    return 0;
}
```
